

**Predicting APS Failures in Scania Trucks:
A Comparative Study of Machine Learning Classifiers on the
IDA2016 Challenge Dataset**

Ayham M Abbad

CSCI 460 - Introduction to Applied Machine Learning

Final Project

Submission Date: May 4th, 2026

Abstract

Binary classification of trucks' Air Pressure System (APS) related failures, under a severe class imbalance and asymmetric misclassification cost, where $FP=10$, and $FN=500$. Scania APS Failure dataset contains approximately 60,000 rows of training instances, and around 170 anonymized numeric features. High missingness rate required median imputation, high skewness required scaling for linear models. Six classifiers trained and tuned using GridSearchCV with stratified 3-fold Cross Validation sections, and threshold tuned top models. The results after tuning indicate that the best model is XGBoost with a total cost of 9900 with a threshold of 0.35. Other metrics for the best model include recall at 0.99, precision at 0.33, and accuracy of 0.95. The information occurred indicate that the family of tree-based models will perform best for this cost-sensitive, imbalanced problem.

Table of Contents

Abstract.....	2
Table of Contents	3
1. Introduction	5
1.1 Problem Definition	5
1.2 Dataset Description	5
1.3 Objectives and Research Questions.....	6
1.4 Report Structure.....	6
2.1 Summary Statistics	6
2.2 Target Variable Distribution.....	7
2.3 Missing Value Analysis.....	8
2.4 Feature Distributions	9
2.5 Class-Conditional Feature Behavior.....	10
2.6 Outliers and Data Quality	10
2.7 Key EDA Takeaways	10
3.1 Data Preprocessing	11
3.1.1 Target Encoding	11
3.1.2 Handling Missing Values	11
3.1.3 Feature Scaling	11
3.1.4 Handling Class Imbalance	11
3.2 Feature Engineering and Selection.....	12
3.3 Machine Learning Models.....	12
3.3.1 Logistic Regression	12
3.3.2 K-Nearest Neighbors	12
3.3.3 Decision Tree.....	12
3.3.4 Random Forest.....	12
3.3.5 Gradient Boosting.....	12
3.3.6 XGBoost	13
3.4 Experimental Setup	13
3.4.1 Data Splitting Strategy	13
3.4.2 Software and Libraries	13
3.5 Hyperparameter Tuning.....	13

3.6 Evaluation Metrics.....	14
3.6.1 Metric Definitions	14
3.6.2 Justification of Metric Choice	15
3.6.3 Confusion Matrix.....	15
4. Results and Discussion	15
1.1 Baseline (Untuned) Model Performance	15
4.2 Tuned Model Performance	16
4.3 Impact of Hyperparameter Tuning	17
4.4 ROC Curves.....	18
4.5 Threshold Tuning on Top Models	19
4.6 Best Model: Detailed Analysis	20
4.6.1 Confusion Matrix.....	20
4.7 Critical Discussion.....	20
4.8 Limitations.....	21
5. Conclusion	21
5.1 Summary of Findings	21
5.2 Answers to Research Questions	22
5.3 Future Work.....	23
References	24

1. Introduction

Predictive maintenance plays an important role in heavy trucks fleets as unexpected failures can lead to costly consequences that sometimes include the live of the driver. By identifying signs of failure before it happens, operators can efficiently schedule timely maintenance services that reduce the unnecessary repairs. The Air pressure system (APS) is a quality target for such an analysis because it is essential for safe operations on the vehicle especially ones related to braking. Since an APS failures can lead to serious operational safety hazard, predicting them can result in a clear outcome of reduction in operations cost and reliability.

1.1 Problem Definition

The prediction task in this project is a binary classification problem in which each truck record must be assigned to one of two classes: APS failure (pos) or non-APS failure (neg). In other words, the model is trained to determine whether the observed sensor and operational measurements indicate a failure related specifically to the Air Pressure System. Two problems observed make such a problem appear difficult and strongly affect how the modeling process should be approached. First, a severe imbalance in the data set, with close to 98.33% of records belonging to the negative class and only 1.67% belonging to the positive class. This means any model could achieve supreme accuracy just by mostly predicting the majority class while still failing to detect the cases that matter most. Second, the problem has a strongly asymmetric misclassification cost defined by the challenge: Cost 1 = 10 for a false positive and Cost 2 = 500 for a false negative, meaning that missing a true failure is 50 times more costly than incorrectly flagging a healthy truck. Because of this combination of extreme class imbalance and uneven error cost, standard accuracy modeling is not the appropriate approach. Instead, the modeling choices has to emphasize methods that are sensitive to minority class detection, use evaluation metrics different than typical accuracy, and account for a much higher penalty associated with false negatives.

1.2 Dataset Description

The dataset used in this project is organized into separate training and test files, the training file contains 60,000 training records, and the test file contains 16,000 test records [1], [2]. The data includes roughly 170 anonymized numeric features. The target is a binary column named 'Class'. The features represent operational and sensor related measures collected from heavy trucks, with grouped names combined of two letters followed by numbers (e.g. aa_01). Their exact meanings are intentionally hidden for competitive and proprietary reasons, so the analysis has to focus on statistical patterns rather than interpretation of individual variables. The target variable defines weather a record was associated to an APS failure or not, pos indicates a failure related to the Air Pressure System, while neg indicates failures caused by other components. One important detail about the data set is data quality, missing values in this dataset were stored as the string 'na'. In addition, 'na' is distributed very unevenly across the features, with some columns containing few missing data and others containing a substantial amount, as a result, preprocessing and imputation is a necessary part of the pipeline.

1.3 Objectives and Research Questions

The objective behind the project is to develop models that effectively classifies trucks based on given parameters to either have a failure related to the Air Pressure System or a failure related to another system component. As the dataset is highly imbalanced and the system must assign a much higher penalty to undetected APS failures than false alarms the goal is not to maximize accuracy, but it is to identify choices when modeling that led to minimize the overall cost of prediction.

To perform this analysis, the report addresses the following. What techniques can we use to challenge the imbalance in the data? And which one works best? Which of the candidate classifiers performs the best with the task defined to minimize the total cost? How much does hyperparameter tuning improves a model's performance over the untuned version of it? By how much does tuning the decision threshold minimizes the cost of a prediction?

1.4 Report Structure

This report begins by a dive into Exploratory data analysis where fundamental properties of the data such as missing values and skewed numbers will be discussed. Then it will move to details of the methodology followed to process the data, feature engineering and the choice made regarding it, the models chosen for the sake of comparison, experiment setup to streamline the process, and finally hyper parameter tuning in order tighten up the models selected. After that, the report presents the results obtained from the different models and discusses their performance based on the evaluation metrics and the challenge-defined cost function. Special attention is given to comparing baseline models, tuned models, and the effect of decision threshold adjustment on the final outcomes. The report then closes with a conclusion that summarizes the main findings, answers the research questions, and reflects on the overall effectiveness of the modeling approach.

2. Exploratory Data Analysis

Exploratory Data Analysis (EDA) is the process scoped to decisions that directly affect the modeling choices. It involves but is not limited to target imbalance, missing data, feature scaling and skewed data, and correlations. This process is crucial for Machine Learning as it enables the engineer to understand the data he's dealing with and in return make choices that positively affect the ending product.

2.1 Summary Statistics

Table 1: Summary Statistics for Some Features

	count	mean	std	min	25%	50%	75%	max
aa_000	60000.0	5.933650e+04	1.454301e+05	0.0	834.0	30776.0	48668.0	2.746564e+06

	count	mean	std	min	25%	50%	75%	max
ab_000	13671.0	7.131885e-01	3.478962e+00	0.0	0.0	0.0	0.0	2.040000e+02
ac_000	56665.0	3.560143e+08	7.948749e+08	0.0	16.0	152.0	964.0	2.130707e+09
ad_000	45139.0	1.906206e+05	4.040441e+07	0.0	24.0	126.0	430.0	8.584298e+09
ae_000	57500.0	6.819130e+00	1.615434e+02	0.0	0.0	0.0	0.0	2.105000e+04
af_000	57500.0	1.100682e+01	2.097926e+02	0.0	0.0	0.0	0.0	2.007000e+04
ag_000	59329.0	2.216364e+02	2.047846e+04	0.0	0.0	0.0	0.0	3.376892e+06
ag_001	59329.0	9.757223e+02	3.420053e+04	0.0	0.0	0.0	0.0	4.109372e+06
ag_002	59329.0	8.606015e+03	1.503220e+05	0.0	0.0	0.0	0.0	1.055286e+07
ag_003	59329.0	8.859128e+04	7.617312e+05	0.0	0.0	0.0	0.0	6.340207e+07

Noticed in the table above is that the data cells have varying scales across the data set columns. This detail matters as for linear models this widespread scale can highly affect the evaluation as the models might consider the columns with larger scale of more importance than the ones with lower values which we know is not be true for an anonymized dataset.

2.2 Target Variable Distribution

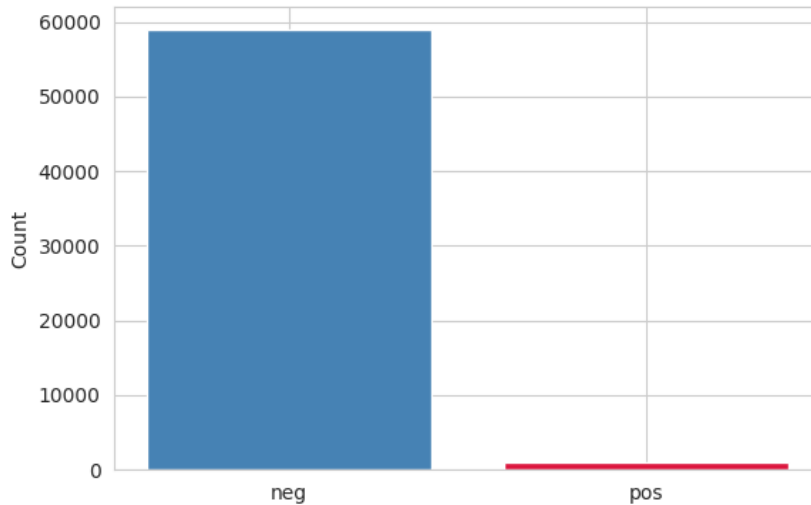


Figure 1: Class Distribution in Training Set

The target variable is the ‘class’ which is a binary being either ‘pos’ or ‘neg’. Looking at the bar chart above, we observe the critical imbalance in the target column with the ‘neg’ dominating the instances ratio at [98.3: 1.6]. This in mind, we can conclude that accuracy alone is not a trustworthy metric as the model can get 0.983 just by predicting ‘**neg**’ for every instance.

2.3 Missing Value Analysis

Missing values are a normal part of any real-world dataset, including the one we are working with. Missing values here are encoded in the raw CSV file as the literal string ‘na’. These missing values were replaced with NumPy’s format of a missing value ‘np.nan’ before any

analysis.

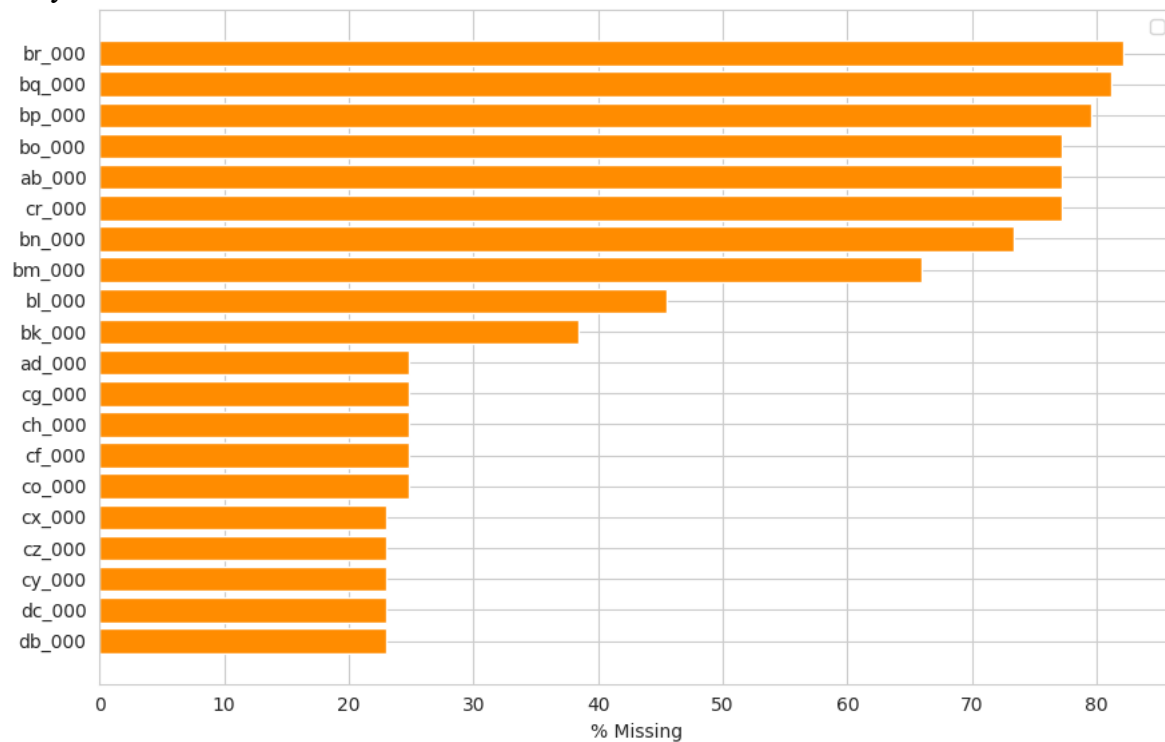


Figure 2: Top Columns by Missing Values Percentage

Above is a bar chart of the top 20 columns with missing values. We can derive from the above the need for a pipeline that applies median imputation inside the pipeline and possibly drops the columns with missing values percentage above 60%.

2.4 Feature Distributions

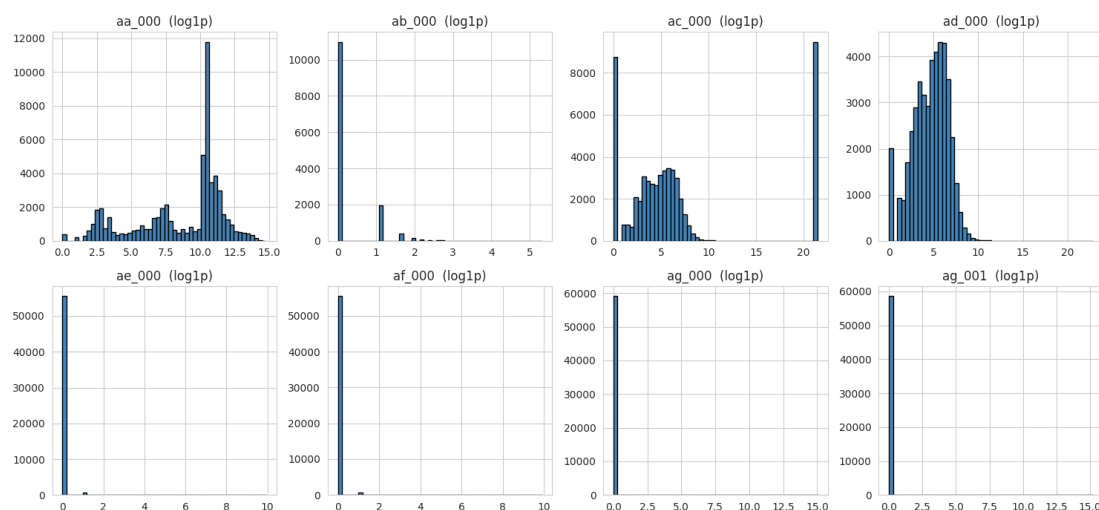


Figure 3: Representative Features Histograms

Most representative features in the histogram above are heavy right skewed counters thus $\log_1 p$ ($\log(x+1)$) was used to scale them in order to display the values normally.

2.5 Class-Conditional Feature Behavior

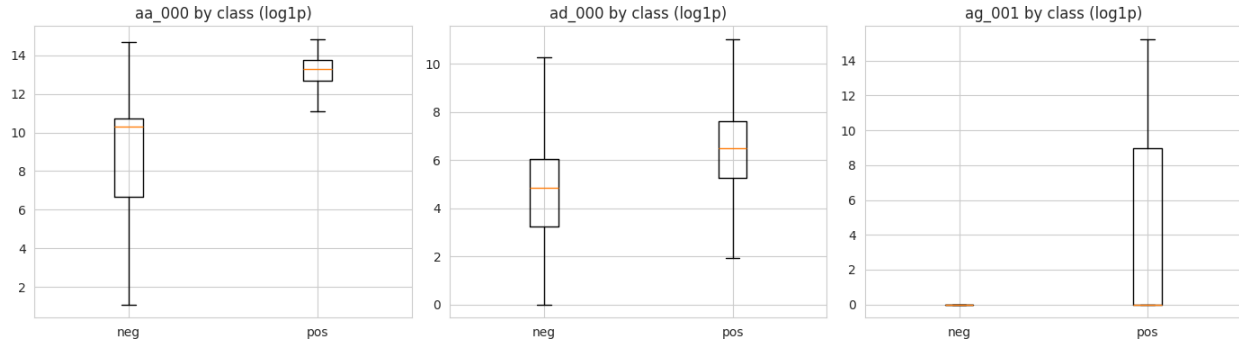


Figure 4: Features Scaled Boxplots

Above is a box plot of scaled features that compares 'pos' vs 'neg'. It's visible that a separation of values between the classes exists for many features, concluding there is a learnable pattern from the features that should be better than majority class guessing.

2.6 Outliers and Data Quality

Analyzing the data on hand, we observe the extreme right tail values that might skew the mean right. In such cases we can either, get rid of those values, or keep them. Determining that depends on questioning the benefit from of them. In our case, it is better to keep them because of many reasons. In the data behavior we observed that having one measure outlying can be highly correlated to the class being neg or pos, this mean it is a pattern that is learnable and can benefit the performance of a model.

2.7 Key EDA Takeaways

The general EDA takeaways are the following:

1. Severe imbalance [98.3: 1.6] requires Weighing classes, or SMOTE.
2. Several columns exceed 70% missing; median imputation is non-negotiable.
3. Heavy right-skew in counters, linear models require scaling, however trees are fine.
4. Visible class separation in several features confirms learnable signal exists.

3. Methodology

In this project the methodology is organized into the following parts; preprocessing, model

selection, experimental setup, hyperparameter tuning, and evaluation metrics. Every Design choice in the pipeline is supported and justified by the EDA's findings

3.1 Data Preprocessing

Starting by encoding 'neg' and 'pos' values. Then moving to creating a pipeline that handles missing values, scales features if needed, and trains the model. After which comes hyperparameter tuning using cross-validation, that combined with a pipeline prevents data leakage into the testing set as pipelines only fit on training folds. Tuning the threshold after leads to further control which benefits our specific grading criteria.

3.1.1 Target Encoding

The target class has two values: 'pos' and 'neg'. The two options are encoded as follows: neg → 0, pos → 1. The target is already binary, and because of that, no one-hot encoder is needed for the translation.

3.1.2 Handling Missing Values

As mentioned prior, some columns in the data set contain a significant number of missing values denoted as the string 'na' prior to the transformation to np.nan. In such situations, two options to substitute are typically standout, either replacing them with the median, or with the mean. Deciding that depends on what is observed during EDA; for our dataset, we observed heavy right-skew in many places. The mean approach is sensitive to skew, because of that it was not chosen. On the other hand, the median approach is robust and not affected by the skew, which makes it a perfect choice for the data on hand.

3.1.3 Feature Scaling

Pipeline is an approach where the steps to process data are chained together into one object which automates the machine learning flow. For this project, we created two pipelines; the reason behind that is the different models that we are training. First is the pipeline under 'make_pipeline_scaled' method, used for linear models such as: Logistic Regression and KNN. It starts with median imputation then standard scaling as such models are sensitive to feature scales. Second is the pipeline under 'make_pipeline_tree' method, used for tree-based models such as: Decision Tree, Random Forest, Gradient Boosting, and XGBoost. It only imputes with the median and does not scale as tree-based models are scale insensitive.

3.1.4 Handling Class Imbalance

Class imbalance is a common problem faced when dealing with real-world data sets. There are many ways to deal with this problem. SMOTE, ROS, RUS, and Class Weights are all methods of dealing with the problems [3], [4]. Class weight is the method that was used for our type of problem. Class weights is an algorithm level approach that address the imbalance. It is superior to all the other methods as it directly modifies the loss function of the model which results in

directly punishing the model harder when it misses a positive which aligns with the overall goal of minimizing the cost [5].

3.2 Feature Engineering and Selection

Feature engineering is the process of deriving better features from already existing ones to better direct and train the model. The data on hand includes features which are fully anonymized, this means that no domain driven feature engineering is needed. Another way to better the features is to reduce dimensions which is not the best choice for our data as it involves high missingness and extreme skews. Applying dimension reduction for our data risks interpretability of feature importances.

3.3 Machine Learning Models

3.3.1 Logistic Regression

Logistic Regression is the linear baseline model; it is sensitive to features scale, it finds the best-fitting S-shaped curve to separate classes, predicting values between 0 and 1, it defaults to 0.5 threshold [6].

3.3.2 K-Nearest Neighbors

K-Nearest Neighbors is a model that works by calculating the distance between data points to find the closest neighbors [7]. It does not make assumptions about the distribution or structure of the data, meaning it does not draw a shape between classes making it a flexible model. Because it is a non-parametric model that relies on distance, it requires scaling. The downside for this model is that it will be slow for the large dataset that we have.

3.3.3 Decision Tree

Decision Tree is a single CART tree model that splits the data into branches based on feature values to make predictions [8]. It is highly interpretable since the decision rules can be visualized directly, but it suffers from high variance meaning small changes in the data can lead to very different trees. It does not require scaling because it is not distance-based. We include it as a baseline for the ensemble methods.

3.3.4 Random Forest

Random Forest is a bagged ensemble model that builds many decorrelated decision trees and averages their predictions [9]. It handles the missing then imputed features well and it is robust to feature scale, so it does not require scaling. By averaging many trees, it naturally reduces the variance of the individual trees making it more stable than a single Decision Tree.

3.3.5 Gradient Boosting

Gradient Boosting is a boosted ensemble model that builds shallow trees sequentially, where each new tree fits the residuals of the previous ones [10]. It is usually stronger than Random Forest on tabular data, but the downside is that it is slower to train and it requires tuning of the learning rate, tree depth, and number of estimators. Like other tree based models, it does not require scaling.

3.3.6 XGBoost

XGBoost is a regularized version of Gradient Boosting that uses the `'scale_pos_weight'` parameter to handle class imbalance [11]. It is typically the strongest baseline for tabular data and it is expected to be competitive for the lowest `'total_cost'` based on our notebook analysis. It is also a tree based model so it does not require scaling.

3.4 Experimental Setup

3.4.1 Data Splitting Strategy

The tabular data for this project is pre-split into two datasets, first dataset is 60,000 rows for training, second dataset is 16,000 for testing; because of that, no additional splitting was done. Within the training set, a 3-fold StratifiedKFold cross validation is used inside GridSearchCV. StratifiedKFold is a splitting technique that preserves the percentage of each class in each fold. The test set is held and only used for the final comparison.

3.4.2 Software and Libraries

Environment: Google Colab

Language: Python 3.x

Data Handling: pandas, numpy [12], [13]

Machine Learning: scikit-learn (Pipelines, SimpleImputer, StandardScaler, StratifiedKFold, GridSearchCV, classification models, and metrics) [14]

Advanced Gradient Boosting: xgboost (XGBClassifier)

Data Visualization: matplotlib, seaborn [15], [16]

3.5 Hyperparameter Tuning

Tuning a model is a crucial process in order to come out with the best possible result. The method followed here involved a 3-fold stratified fold cross-validated grid search where scoring was based on the recall metric, then refitting the best setting for recall [17]. The reason behind recall as CV scoring target is that it correlates with lower False Negative count which has a 50x cost ratio, meaning lower FN = lower cost.

Table 2: Models Hyperparameters

	Model	Tuned Hyperparameters
0	Logistic Regression	$C = \{0.01, 0.1, 1, 10\}$, $class_weight = \{None, balanced\}$
1	KNN	$n_neighbors$, $weights = \{uniform, distance\}$, $metric = \{euclidean, manhattan\}$
2	Decision Tree	$max_depth = \{5, 10, 20, None\}$, $min_samples_split = \{2, 10, 20\}$, $class_weight = \{None, balanced\}$
3	Random Forest	$n_estimators = \{100, 200\}$, $max_depth = \{10, 20, None\}$, $min_samples_split = \{2, 10\}$, $class_weight = \{balanced, balanced_subsample\}$
4	Gradient Boosting	$n_estimators = \{100, 200\}$, $learning_rate = \{0.05, 0.1\}$, $max_depth = \{3, 5\}$
5	XGBoost	$n_estimators = \{100, 200\}$, $max_depth = \{3, 5, 7\}$, $learning_rate = \{0.05, 0.1\}$, $subsample = \{0.8, 1.0\}$, $colsample_bytree = \{0.8, 1.0\}$

3.6 Evaluation Metrics

3.6.1 Metric Definitions

Accuracy is a metric used to measure the percentage of correct predictions made by the model. It is calculated as:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision is a metric that measures the accuracy of positive predictions, in other words, how many predicted positives were true. It is calculated as:

$$Precision = \frac{TP}{TP + FP}$$

Recall is a metric that measures the percentage of actual positives that were correctly classified by the model. It is calculated as:

$$Recall = \frac{TP}{TP + FN}$$

F1-score is the metric that combines precision and recall into one metric. It's calculated as:

$$F1 = 2 * \frac{Precision * Recall}{Precision + Recall}$$

ROC-AUC is graphical plot that shows how well the model separates the positive and negative classes across different threshold values. A higher curve means the model is better at ranking Positive cases above Negative cases [18], [19].

Finally, a custom performance metric was created because having a FN is much more expensive than having an FP however, we still need to make sure we don't miss classify all the negatives. It is calculated as:

$$\text{Total Cost} = 10 * FP + 500 * FN$$

3.6.2 Justification of Metric Choice

Out of the above metrics, two are the most important. Recall is one as it directly reduces FN which has the highest cost. Total cost is the second as our goal is to minimize the overall cost which is an overall operationally correct objective. The decision here is influenced by the imbalance and asymmetric cost. Accuracy is only reported here to report the inadequacy of it, and the ROC-AUC is included as an independent summary.

3.6.3 Confusion Matrix

Confusion matrix is a table that shows the intersection of actual and predicted classes. It is produced for the best model on the test set and is presented in section 4 of the code.

4. Results and Discussion

The results for our models trained are presented in four parts. First, the untuned baselines metrics acquired from testing set are presented to compare performance. Second, the tuned models metrics evaluated from the testing set is presented in a similar format to the previous. Third, the top models are taken out for further tuning to get the best threshold of choice. And finally, further analysis of the best model is done by showing the confusion matrix with the threshold chosen and feature importance.

1.1 Baseline (Untuned) Model Performance

Table 3: Metrics Table Prior to Tuning

	accuracy	precision	recall	f1	roc_auc	TN	FP	FN	TP	total_cost
LogReg	0.974938	0.481894	0.922667	0.633120	0.979349	15253.0	372.0	29.0	346.0	18220.0
KNN	0.987563	0.869748	0.552000	0.675367	0.919784	15594.0	31.0	168.0	207.0	84310.0
DecisionTree	0.986062	0.750000	0.608000	0.671576	0.801568	15549.0	76.0	147.0	228.0	74260.0

RandomForest	0.989125	0.938865	0.573333	0.711921	0.991944	15611.0	14.0	160.0	215.0	80140.0
GradBoost	0.990688	0.897887	0.680000	0.773900	0.992285	15596.0	29.0	120.0	255.0	60290.0
XGBoost	0.991812	0.797561	0.872000	0.833121	0.993970	15542.0	83.0	48.0	327.0	24830.0

Looking at Table 2, we observe an unanticipated result. Logistic Regression model outperformed all models including tree-based ones. This was not expected as tree-based models typically outperform linear models on tabular data with skews and imbalances. Logistic regression is observed to have the highest recall rate which results in it being sensitive to positive cases, it's success here is found in the way it was trained with class weights being balanced. It achieved this result out regarding it having the lowest f1-score and generating more FP. Followed by Logistic Regression is Extreme Gradient boost with a total cost of 24830.

4.2 Tuned Model Performance

Table 4: Metrics Table After Tuning

	best_cv_recall	accuracy	precision	recall	f1	roc_auc	TN	FP	FN	TP	total_cost
XGBoost	0.947004	0.962375	0.381648	0.976000	0.548726	0.993618	15032.0	593.0	9.0	366.0	10430.0
LogReg	0.906008	0.976500	0.499289	0.936000	0.651206	0.989751	15273.0	352.0	24.0	351.0	15520.0
Decision Tree	0.924988	0.958250	0.352170	0.930667	0.510981	0.951349	14983.0	642.0	26.0	349.0	19420.0
Random Forest	0.869013	0.978000	0.517774	0.893333	0.655577	0.990783	15313.0	312.0	40.0	335.0	23120.0
GradBoost	0.710986	0.992188	0.908497	0.741333	0.816446	0.994261	15597.0	28.0	97.0	278.0	48780.0

KNN	0.50799 9	0.988 688	0.907 563	0.576 000	0.704 731	0.943 657	1560 3.0	22. 0	15 9.0	21 6.0	79720 .0
------------	--------------	--------------	--------------	--------------	--------------	--------------	-------------	----------	-----------	-----------	-------------

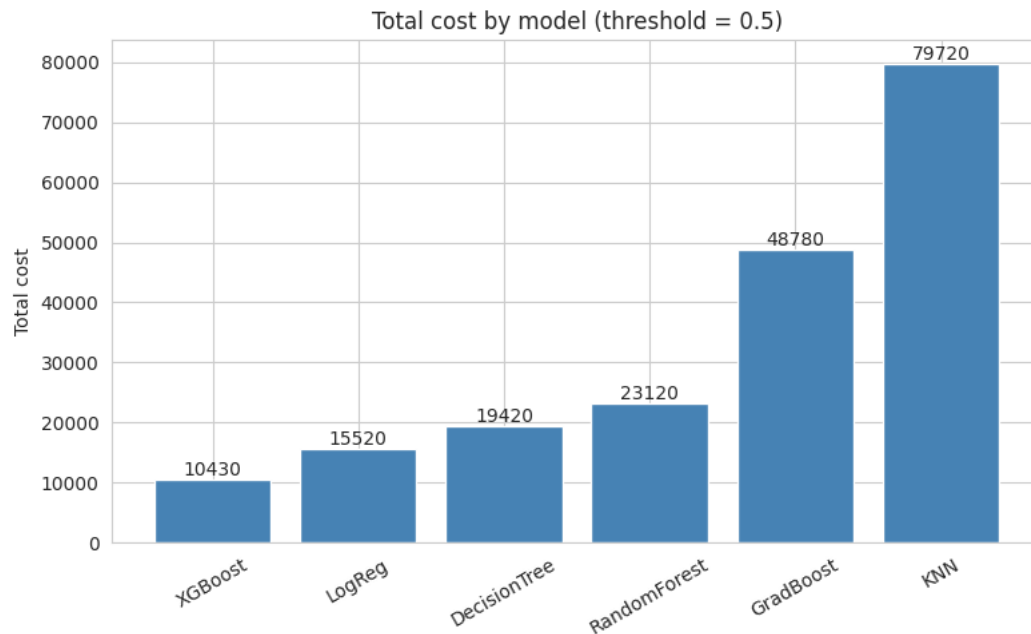


Figure 5: Total Cost for models with 0.5 Threshold

Observed Above is the change of ranking models after tuning. XGBoost is the first in ranks with 0.94 cross validated recall and 0.97 testing set recall. Followed by which is the Logistic Regression which previously ranked the first before tuning. Noticed from the previous is that recall metric is the single handedly the most important standard metric to minimize the cost.

4.3 Impact of Hyperparameter Tuning

Table 5: Recall & Total Cost Difference

Model	Delta Recall	Delta Total Cost
-------	--------------	------------------

XGBoost	+0.104000	-14,400
LogReg	+0.013333	-2,700
DecisionTree	+0.322667	-54,840
RandomForest	+0.320000	-57,020
GradBoost	+0.061333	-11,510
KNN	+0.024000	-4,590

Above are the differences in the metrics observed to be determinant of performance between Untuned and tuned models. We can see that some of the models befitted greatly from tuning like Decision Tree, Random Forest, and XGBoost. On the other hand, some did not change much like the Logistic Regression and KNN. Though LogReg provided a strong base line off the start, tuning tree based models yielded higher returns as trees can handle complex decisions for imbalanced data.

4.4 ROC Curves

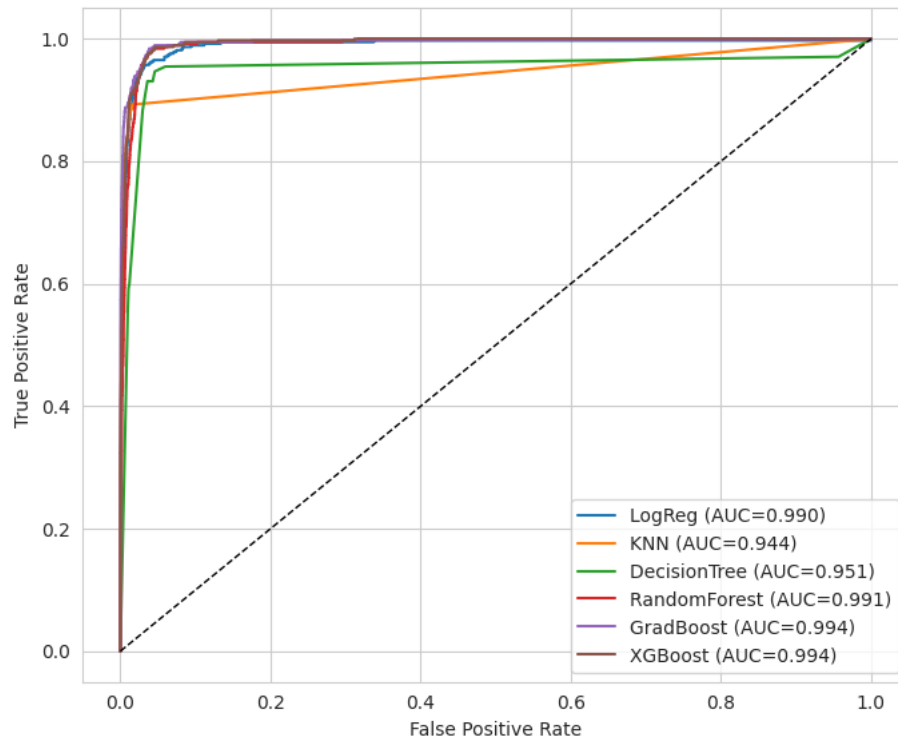


Figure 6: ROC Tuned Models Curve

The ROC curve plot above shows all six tuned models and their Area Under the Curve (AUC). The models that dominate this region can successfully identify most positive cases early on without triggering an overwhelming number of false alarms. We observe that AUC score does not necessarily agree with the total cost as the AUC measures the ability of a model to correctly classify across all possible thresholds weighing the graph as a whole equally.

4.5 Threshold Tuning on Top Models

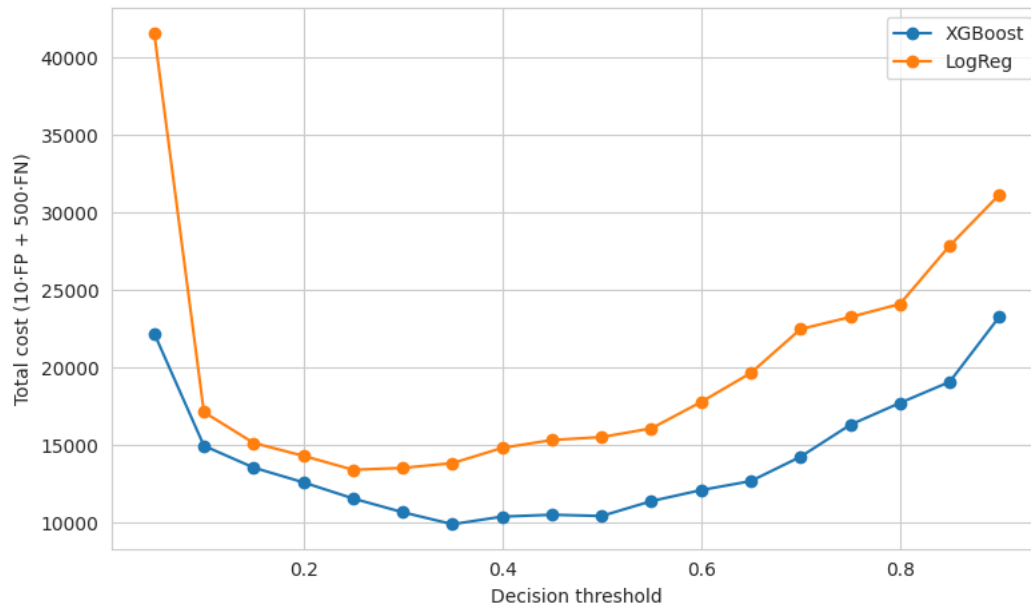


Figure 7: Total cost V.S. Decision Threshold

The figure above shows how the total cost shifts when the thresholds are changed and this is exactly how threshold tuning was done. Threshold sweep methodology is a post processing technique that involves moving the thresholds to maximize or minimize a specific performance metric, Total Cost minimization in our case.

4.6 Best Model: Detailed Analysis

4.6.1 Confusion Matrix

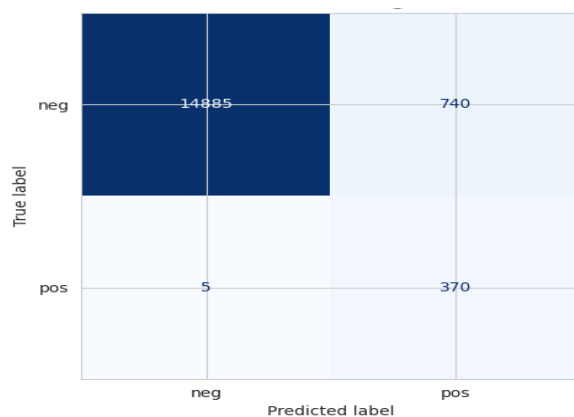


Figure 8: Confusion Matrix for XGBoost with 0.35 Threshold

4.7 Critical Discussion

Looking at XGBoost, Why did it perform the best? In order to answer such a question, we have to dive into how XGBoost works. XGBoost is an optimized version of gradient boosting. Both models work by building trees one after another, where each new tree attempts to correct the errors made by the previous trees. The difference is that XGBoost includes additional improvements that make the model more effective and efficient. These improvements include regularization, faster computation, better handling of missing values, and more tuning options. Because of these additions, XGBoost is often expected to perform better than standard gradient boosting, especially when working with structured tabular data that contains imbalance, skewed features, and missing values.

One surprising outcome observed from the results is that the linear model performed better than KNN. This was not fully expected because KNN can sometimes capture more flexible patterns than a linear model. However, in this dataset, KNN may have been negatively affected by the high number of features, missing values, and the imbalance between the two classes. Since KNN depends on distance between observations, the model can struggle when many features are noisy or scaled differently. Logistic Regression, on the other hand, benefited from class weighting, which made it more sensitive toward the positive class and helped reduce false negatives. Another interesting result is that the tuned Decision Tree approached the performance of Random Forest. This can be explained by the fact that tuning helped control the tree depth and reduce overfitting, allowing the single tree to make more balanced decisions. However, Random Forest still has the advantage of combining many trees, which usually makes it more stable and less dependent on one specific split in the data.

4.8 Limitations

KNN is the model that faced the most limitations, it starts with the imbalanced data that was not addressed for the model as its version in SKLearn does not include a parameter to balance classes weights and no action was taken to help that in the preprocessing step. SMOTE was considered initially but was not acted upon as it introduces many problems. First is the increased size of training data from 60,000 to 118,000 rows which increases the time of training significantly. Second, the data on our hands has severe missingness and was imputed using median strategy, creating synthetic samples in such a case will generate truck sensor measures that are not realistic to real world.

Cross Validation used 3 folds and not more as it reduces training time this way especially for the KNN model as it took a lengthy time even with only trying 11 neighbors as the only parameter to try.

Features as we know were anonymized, this led to the inability to engineer features and columns could not be interpreted semantically or translated into engineering actions.

5. Conclusion

5.1 Summary of Findings

Ultimately, this analysis findings start with the data. The data dealt with had several problems in need of addressing and they are the following. First, data imbalance was observed in the EDA section, at a 98.3 to 1.6 ratio for the negative and positive classes, the data is heavily imbalanced. In addition to the previous, our problem includes asymmetric cost at 500 for FN and 10 for FP, this means finding positive instances is far more important than finding negatives. Second, is data missingness, some features include more than 70 percent missing rows, this might affect the learning process of our models. Because of that, the pipeline was designed to address these issues through median imputation, class imbalance handling, and model-specific preprocessing. After comparing the models before and after tuning, XGBoost was observed to be the winning model. After hyperparameter tuning, XGBoost achieved the strongest balance for this problem with a testing recall of 0.976 and a total cost of 10,430 at the default 0.5 threshold. This result is important because the model only missed 9 positive cases, which matters heavily due to the 500 cost assigned to every false negative. Although XGBoost created more false positives than some other models, this tradeoff is acceptable in this specific problem framing because false positives are much cheaper than false negatives. In other words, the model was not selected because it had the cleanest accuracy or precision, but because it aligned the most with the cost structure of the project.

The improvement from tuning was also significant. Before tuning, XGBoost had a total cost of 24,830, but after tuning it decreased to 10,430, meaning the total cost was reduced by 14,400. Compared to the best untuned model, which was Logistic Regression with a total cost of 18,220, tuned XGBoost still improved the result by 7,790. This shows that tuning was not only improving the model generally, but it changed the ranking of the models by making XGBoost perform better than the strong Logistic Regression baseline. If threshold tuning is applied after this point, the final selected threshold should be compared against this 0.5 threshold version, since 10,430 is the cost that the tuned XGBoost achieved before changing the decision threshold. Ultimately, the main finding is that the best result came from matching the model selection and tuning process to the actual operational cost, rather than relying on accuracy alone.

5.2 Answers to Research Questions

Many techniques can be implemented to challenge the data imbalance. It was concluded that the best technique is algorithm level technique, where we penalize the model harsher for missing a positive than missing a negative. SMOTE was considered, however, because of high missingness and large dataset, it was chosen not to go with.

Prior to tuning we observed that the best model was the logistic regression, which was unanticipated, however after tuning, the results changed and Extreme Gradient Boost scored first with the least overall cost defined. Hyperparameter tuning helped XGBoost to rank first, XGBoost has many parameters that you can customize like learning rate and subsample which help to prevent overfitting. All the models trained benefited from tuning, however, models like KNN befitted conservatively.

Tuning the decision threshold makes for a good final step. It matters as we need to find a final spot to minimize the cost, it includes getting the least amount of FN while also making sure we don't misclassify many negatives as it is still costly. Tuning the threshold was done to the two best models after tuning which are XGBoost and LogReg. The best threshold for XGBoost was

0.35 and resulted in a total cost of 9900. On the other hand, the best threshold for LogReg was 0.25 and resulted in a total cost of 13410.

5.3 Future Work

Future work for this project can start with improving the model structure used. The first direction is applying stacking, where more than one strong model is combined to produce a final prediction. Models such as Random Forest, XGBoost, and Gradient Boosting can be used first, then a simple Logistic Regression meta-learner can be applied over their predictions. The reason behind this is that each model might learn the data in a different way. Random Forest depends on bagging and reduces variance, while XGBoost and Gradient Boosting depend on boosting and learn from previous model errors. Combining these models might allow the final model to take advantage of the different learning patterns, which can result in better recall and possibly lower total cost.

Second, another future work direction is to re-run the full pipeline after dropping the columns that have more than 70 percent missing values. In this analysis, median imputation was used because of the skewness observed in EDA, however, columns with extremely high missingness might still be a problem. These columns may either contain useful information, or they may add noise to the model and affect the learning process negatively. Because of that, removing them and comparing the results with the original pipeline would show how much these high-missingness columns contribute. The comparison should be made based on recall and total cost, because these are the two metrics most connected to the goal of this project. Ultimately, this would help determine whether keeping these features is beneficial, or if a cleaner version of the dataset can achieve the same or even better results.

References

- [1] C. F. Costa and M. A. Nascimento, “IDA 2016 industrial challenge: Using machine learning for predicting failures,” in *Advances in Intelligent Data Analysis XV (Lecture Notes in Computer Science, vol. 9897)*. Cham, Switzerland: Springer, 2016, pp. 381–386.
- [2] T. Lindgren and J. Biteus, “APS Failure at Scania Trucks Data Set,” UCI Machine Learning Repository, 2017. [Online]. Available: <https://archive.ics.uci.edu/ml/datasets/APS+Failure+at+Scania+Trucks>
- [3] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “SMOTE: Synthetic minority over-sampling technique,” *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, Jun. 2002.
- [4] H. He and E. A. Garcia, “Learning from imbalanced data,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, no. 9, pp. 1263–1284, Sept. 2009.
- [5] C. Elkan, “The foundations of cost-sensitive learning,” in *Proc. 17th International Joint Conference on Artificial Intelligence (IJCAI)*, Seattle, WA, USA, Aug. 2001, pp. 973–978.
- [6] D. W. Hosmer, S. Lemeshow, and R. X. Sturdivant, *Applied Logistic Regression*, 3rd ed. Hoboken, NJ, USA: Wiley, 2013.
- [7] T. Cover and P. Hart, “Nearest neighbor pattern classification,” *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, Jan. 1967.
- [8] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification and Regression Trees*. Belmont, CA, USA: Wadsworth, 1984.
- [9] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, Oct. 2001.
- [10] J. H. Friedman, “Greedy function approximation: A gradient boosting machine,” *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, Oct. 2001.
- [11] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD ’16)*, San Francisco, CA, USA, Aug. 2016, pp. 785–794.
- [12] W. McKinney, “Data structures for statistical computing in Python,” in *Proc. 9th Python in Science Conference (SciPy 2010)*, Austin, TX, USA, Jun. 2010, pp. 56–61.
- [13] C. R. Harris et al., “Array programming with NumPy,” *Nature*, vol. 585, no. 7825, pp. 357–362, Sept. 2020.
- [14] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [15] J. D. Hunter, “Matplotlib: A 2D graphics environment,” *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, May/Jun. 2007.
- [16] M. L. Waskom, “seaborn: Statistical data visualization,” *Journal of Open Source Software*, vol. 6, no. 60, p. 3021, Apr. 2021.

- [17] R. Kohavi, “A study of cross-validation and bootstrap for accuracy estimation and model selection,” in Proc. 14th International Joint Conference on Artificial Intelligence (IJCAI), Montreal, QC, Canada, Aug. 1995, pp. 1137–1143.
- [18] T. Fawcett, “An introduction to ROC analysis,” Pattern Recognition Letters, vol. 27, no. 8, pp. 861–874, Jun. 2006.
- [19] J. Davis and M. Goadrich, “The relationship between precision-recall and ROC curves,” in Proc. 23rd International Conference on Machine Learning (ICML ’06), Pittsburgh, PA, USA, Jun. 2006, pp. 233–240.